

ISS Intelligent Reasoning System Practice Module
Master of Technology in Artificial Intelligent System

Data Quality Assurance System for Wire Bonding Inspection

Mar-May 2026

Group Name: Vision Guard

Team Members

Zheng Bo (A0339761U)

Sun Yan (A0340238L)

Abstract

Inspecting wire bonding via Automated Optical Inspection (AOI) is a challenging task. The large amount of image and annotation data are collected to train edge deep learning models for machine deployment and to validate machine software output through continuous integration. The data is subject to image and annotation quality issues resulting in sub-optimal model performance and diminished software reliability. To resolve these inefficiencies, this project implements a hybrid framework that integrates a rule-based expert system, decision tree classifier leveraging handcrafted features, and a finetune Segment Anything Model 3 (SAM3). To handle high-resolution image incompatible with SAM3 input size, the system utilizes a sophisticated partitioning and merging strategy optimized through hierarchical clustering, bin packing, and genetic algorithms. Furthermore, the SAM3's image exemplar recommendation system is developed by collaborative filtering. Data preparation utilizes active learning for precise curation alongside the simulation of quality-compromised data to rigorously validate system performance.

Keywords: AOI, Rule based expert system, Decision tree classifier, Segment Anything Model 3, Hierarchical clustering, Bin Packing, Genetic Algorithm, Exemplar recommendation system, Collaboration based filtering, Active Learning

Table of Contents

1. Introduction	4
1.1 Background	4
1.2 Project Objectives	5
2. Market Research	6
3. System Architecture and Modelling.....	8
3.1 JSON Validator	9
3.2 Feature-based Classifier Validator.....	10
3.3 SAM3 based Validator.....	11
3.3.1 Image Partition and Merge	11
3.3.2 Exemplar Recommender	13
3.3.3 SAM3 Finetune	15
3.3.4 Label Error Identification	15
3.4 Result Merging	15
4. System Implementation	16
4.1 Data Preparation	16
4.1.1 Synthetic Data Generation.....	18
4.2 JSON Validator	18
4.3 Feature based classifier Validator.....	19
4.4 SAM3 based Validator.....	19
4.5 System Evaluation.....	20
4.6 System GUI.....	21
5. Conclusion	22
5.1 Summary.....	22
5.2 Limitations	22
5.3 Future work	23

1.Introduction

1.1 Background

Wire bonding is an interconnection technique using fine metal wires to link silicone die to substrate or lead frame for integrated circuit input and output. Automotive and camera sensor integrated circuits rely on wire bonding for reliable packaging. Because these applications demand extreme precision, any bonding failure—such as a fractured wire or insufficient contact—renders the circuit defective and unacceptable for use in these safety-critical and high-performance systems.

Automatic optical inspection (AOI) machines (See Fig. 1) are widely relied upon to detect wire defects. These systems utilize high-resolution optics to capture precise images of the circuitry, which are then transmitted to dedicated inspection software. To assess the quality of the wires, the software processes these images using a combination of advanced deep learning models and traditional computer vision techniques, such as blob analysis.

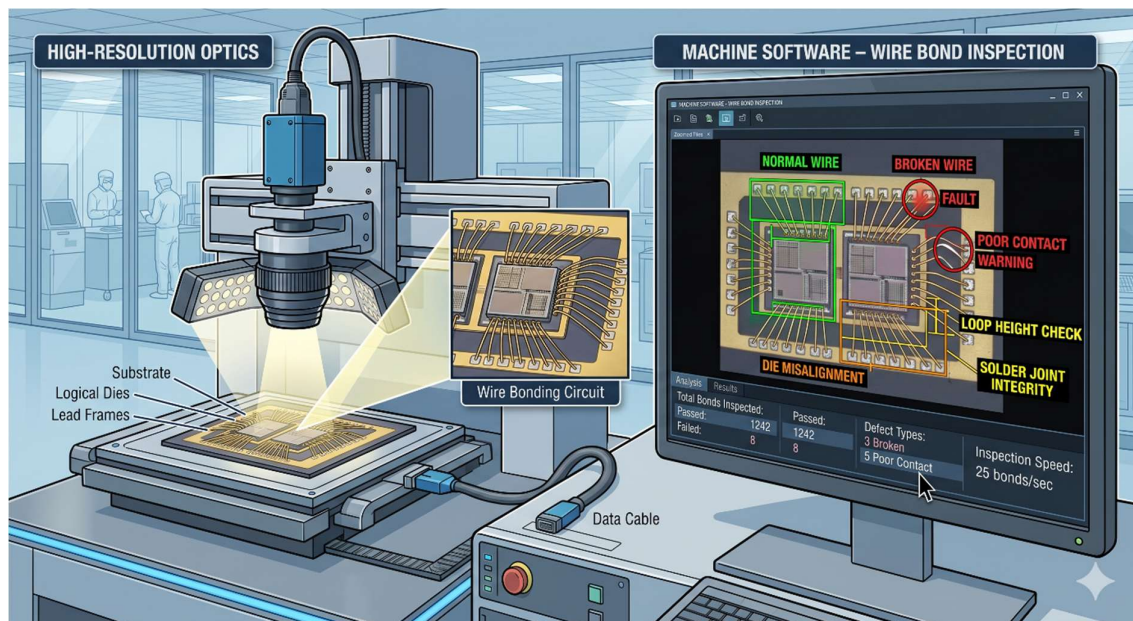


Figure 1. AOI Machine System (Generated by Gemini)

To maintain the AOI system, a large database of images is collected, complete with precise annotations such as wire boundary polygons and bond program defining where the wire should be bonded. Each data point consists of image and JSON annotation file (See Fig 2.).



Figure. 2 Image and JSON annotation file

Because the machine's software undergoes frequent updates for new features and bug fixes, each new release is tested against this database to ensure its inspection results align perfectly with the annotated ground truth. Furthermore, this dataset is critical for training deep learning models such as segmentation, object detection, and classification. Consequently, data errors in the images or annotations will directly cause software inspection errors and model failures in production run.

1.2 Project Objectives

The image data are mainly collected from AOI machine. The annotation are generated by manual annotation or pre-annotation by customized model plus human verification. To detect data errors in image and annotation, we implement a data quality assurance system. A summary of the potential data errors is outlined below:

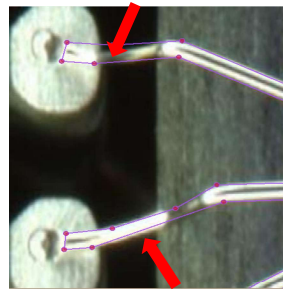
Error	Description	Source
Undefined label	The label name is not “wire”;	Manual annotation
Undefined shape type	The shape type is not “polygon”; the issue arises from errors in manual annotation.	Manual annotation
Shape points out of image range	The “points” field represents a list of polygon vertices, some of which fall outside the image boundaries;	Manual annotation
Dark image	The image is dark;	LED lighting spoiled

Low contrast image	The wire is not visually observable;	Improper settings of LED lightings
Image out of focus	The image is blur;	Package out of focus or in motion when capturing image
Inaccurate wire boundary	The wire boundary does not align with edges in image.	Manual annotation or poor performance of customized model
Inconsistent wire boundary	Different packages have different rules for wire boundary.	Manual annotation by different annotators
Missing annotation	Some wires are not annotated	Manual annotation or poor performance of customized model

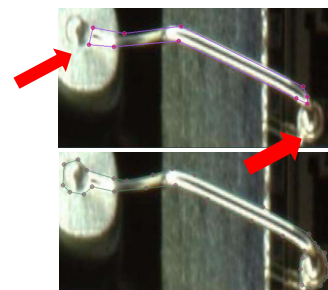
Table 1. Data Errors



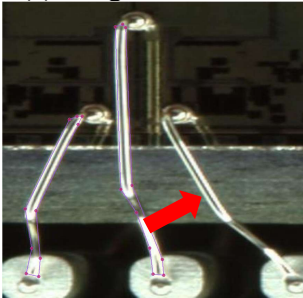
(a) Image out of focus



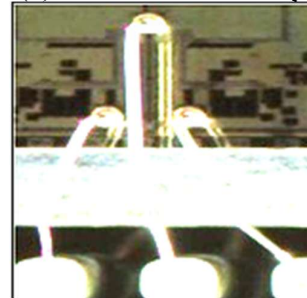
(b) Inaccurate boundary



(c) Inconsistent boundary



(d) Missing annotation



(e) Low contrast



(f) Dark image

Figure 3. Data Errors Illustration

2. Market Research

There exists many software or platform supporting data quality assurance, such as Cleanlab[1], Voxel 51[2], Computer Vision Annotation Tool[3], v7 Darwin[4], Super Annotate[5], etc. The supported methodologies can be categorized into four distinct approaches. The first is multi-annotation method. This approach involves having a single dataset annotated by multiple annotators. Data quality is assessed by measuring the variance between these annotations; high variance typically indicates underlying issues with the data or the annotation instructions. This

method is highly resource-intensive and carries a significant human labour cost. The second is AI-assisted annotation. Customized deep learning models, such as YOLO26[6], are utilized to automate the annotation process. Data points are flagged as problematic if the model predictions exhibit high entropy or demonstrate a significant discrepancy relative to the current annotations. These models often suffer from poor generalization leading to many false alarm for new data. In specialized fields like wire bonding, the performance degrades significantly due to frequent domain shifts. The wire bonding environment is constantly evolving with new alloy materials, advanced coating techniques (e.g., gold-plated silver), and increasing silicon die sizes. These factors alter circuit layouts and wire appearances, rendering static models less effective over time. The third method is feature embedding and clustering. This technique involves mapping 2D images into 1D vectors using advanced embedding models like DINOv2[7]. Once the data is projected into this embedding space, clustering algorithms are applied to group similar data points. Data points positioned significantly far from established cluster centres are identified as outliers or potential annotation errors. While this approach effectively identifies duplicate or anomalous images, it is incapable of detecting precise wire boundary errors.

The last method is using vision foundation model. The latest state-of-the-art vision foundation is SAM3[8] that can detect, segment, or track objects in image and video using geometrics prompt including point, box or mask, text prompt or image exemplars (See Fig. 4).

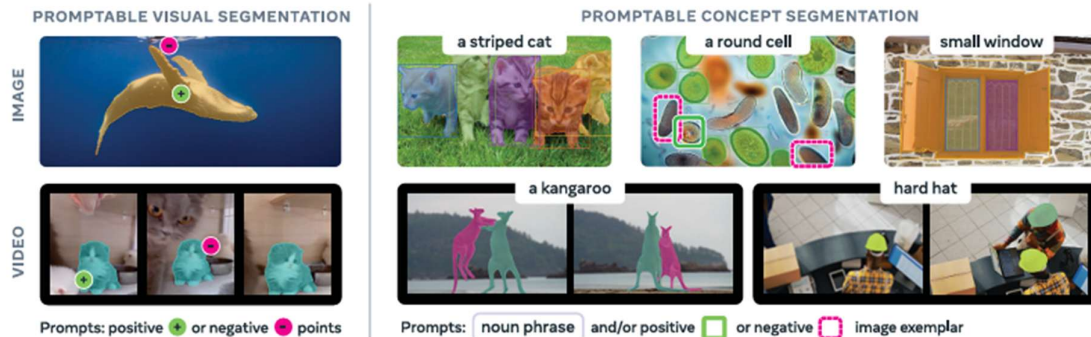


Figure 4. SAM3 (Quoted from SAM3 paper)

While SAM3 demonstrates strong generalization across various semiconductor applications due to its diverse training on medical and multi-domain datasets, it encounters significant hurdles with wire bonding data. The first is unsatisfactory performance on wire bonding dataset. As illustrated in Figure 4, SAM3 often fails to detect certain wires or inadvertently merges masks of adjacent wires. This is primarily due to a distribution shift; the unique appearance of wire bonding data is absent from the training set, causing the detector's region proposal queries to fail. The second is SAM3 accepts 1008 x 1008 color image while the wire bonding image size ranges from 512 x 512 to 5120 x 5120. Standard resizing is not a viable solution here, as the extreme thinness of the wires means high-factor resizing would cause them to disappear entirely. The third is image exemplar. The quality of segmentation results is heavily contingent

on the selection of image exemplars. Currently, these exemplars are restricted to the same image, limiting the model's flexibility.

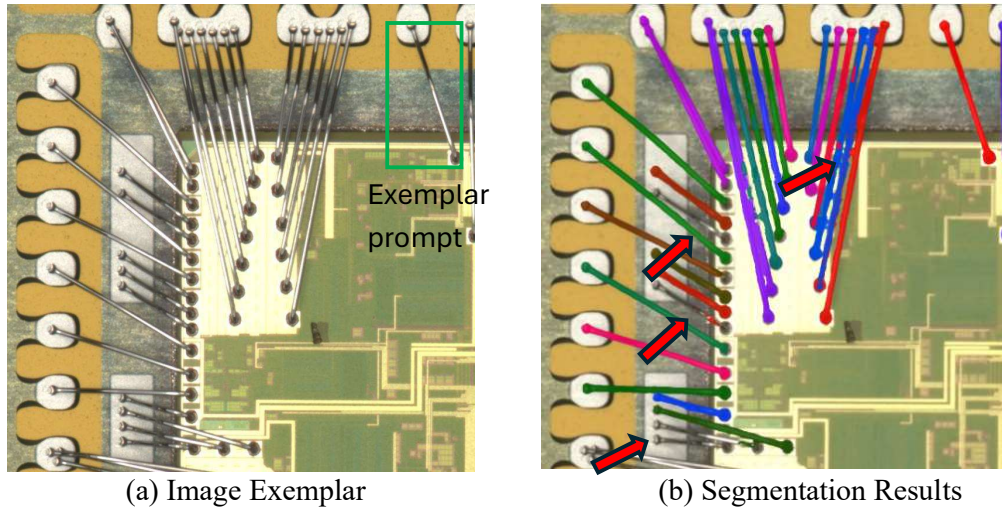


Figure 5. SAM3 Performance on wire bonding data

This project delivers a data quality assurance system using hybrid approach including expert based rule based system, feature based classification and finetuned SAM3. The system is expected to reduce the label cost in data validation and verification and better generalization on new dataset.

3. System Architecture and Modelling

By integrating a hybrid validation architecture, the system ensures high-performance label error detection across both images and their corresponding annotation. The process follows a multi-stage pipeline designed to maximize data integrity. The system processes input pairs of images and JSON annotations through three distinct path including JSON validator, feature-based classifier validator, and SAM3 based validator (See Fig. 6). Rather than relying on a single check, the system merges the outputs from all three validators. This ensemble approach creates a robust filtering mechanism that simultaneously boosts precision (by reducing false positives) and recall (by catching elusive errors), resulting in a highly reliable error detection suite.

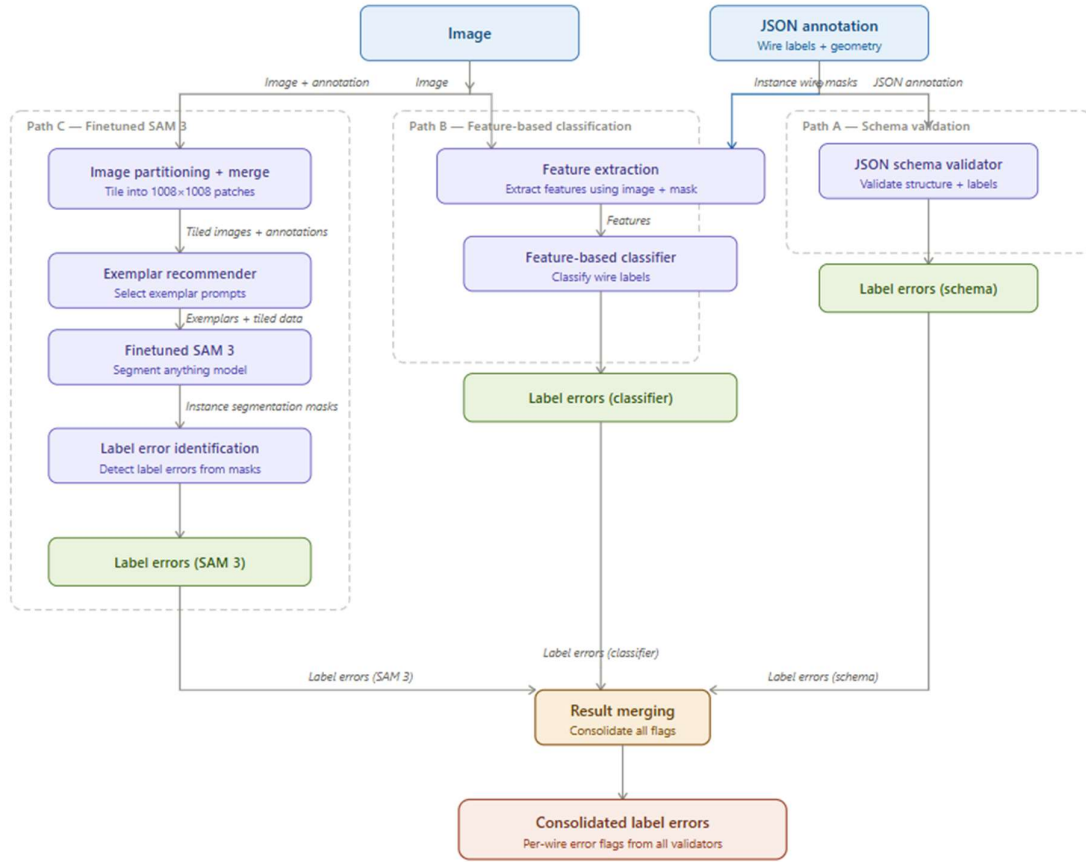


Figure 6. System Architecture

3.1 JSON Validator

The JSON validator conducts structural checks to ensure data formatting conforms to predefined standards. A JSON Schema (See Fig. 7) is implemented to validate label names, shape configurations, point boundaries, and additional metadata attributes.

```

SHAPE_SCHEMA = {
  "type": "object",
  "required": [
    "Type",
    "points",
    "Attributes",
    "properties": {
      "Type": { Defined Shape Type
        "type": "string",
        "enum": [
          "polygon"
        ],
      },
      "points": { Defined Shape Points
        "type": "array",
        "minItems": 1,
        "items": {
          "type": "array",
          "minItems": 2,
          "maxItems": 2,
          "items": {
            "type": "number",
            "minimum": 0,
            "maximum": 5120 }
          },
        },
      "Attributes": { Required Metadata
        "type": "object",
        "required": [
          "category_id"
        ],
        "properties": {
          "category_id": {
            "type": "number"
          }
        },
      },
      "additionalProperties": True
    }
  ]
}

```

Figure 7. Wire Shape Schema

3.2 Feature-based Classifier Validator

The wire features including “stroke width”, “stroke length”, “hue”, “hue standard deviation (SD)”, “saturation”, “saturation SD”, “intensity” and “intensity SD” are extracted from image and wire mask generated from JSON annotation.

The stroke width and length are length and thickness of the wire mask. Using the `medial_axis` function from the `scikit-image` library [9], we extract the medial axis and the corresponding distance transform of the mask. The distance transform generates a map where each pixel value represents the Euclidean distance to the nearest boundary. Consequently, the stroke width is

derived by averaging the distance values along the medial axis, while the stroke length is determined by the total pixel count of the medial axis.

The input image is loaded as $H \times W \times 3$ in RGB format representing red, green, and blue colour information. Upon converting the image to the HSI (Hue, Saturation, Intensity) colour space, we extract the mean values for each component by averaging the HSI pixels within the wire mask. Additionally, the corresponding standard deviations are calculated to characterize colour variance.

These features are evaluated via histogram analysis to determine their discriminative power in distinguishing between normal and erroneous labels. Features lacking significant variance or separation are discarded. Subsequently, a decision tree classifier is trained on the remaining feature set to automate label error detection.

3.3 SAM3 based Validator

The SAM 3-based validator processes high-resolution input images (exceeding 1008×1008) by partitioning them into smaller patches. These patches are then efficiently arranged into 1008×1008 bin images for batch processing. These bins are processed by a SAM 3 model fine-tuned on a specialized wire bonding dataset. The resulting instance masks are cross-referenced with the original wire annotations; any significant geometric discrepancies between the predictions and the annotations are flagged as label errors.

3.3.1 Image Partition and Merge

To optimize inference efficiency, an image partitioning and merging strategy is employed to minimize the total number of inputs processed by SAM 3. The Bond program provides the line segments (Fig. 8) necessary for image partitioning. In real-world datasets, these segments may not perfectly align with the physical wires due to inherent variations in the wire bonding.

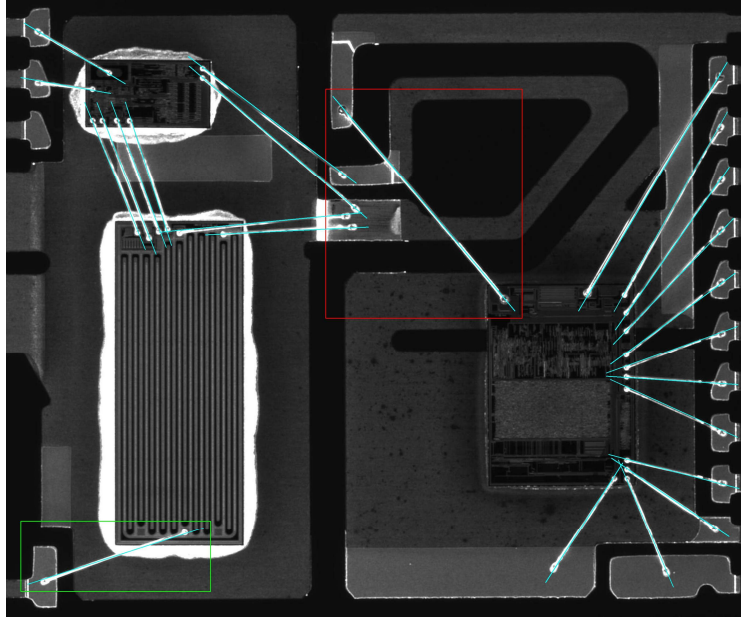


Figure 8. Bond Program

First, the bounding boxes for all line segments are extracted from the annotations. The box is expanded by certain size considering wire bonding variations. These boxes are then grouped via hierarchical clustering, using a custom dissimilarity function and linkage criteria designed to cluster highly overlapping geometries. The resulting cluster tree is traversed using a Breadth-First Search (BFS); if a node's bounding dimensions exceed the 1008x1008 threshold, the search continues to its children. Once a node fits within the size constraints, it is finalized as an individual image patch.

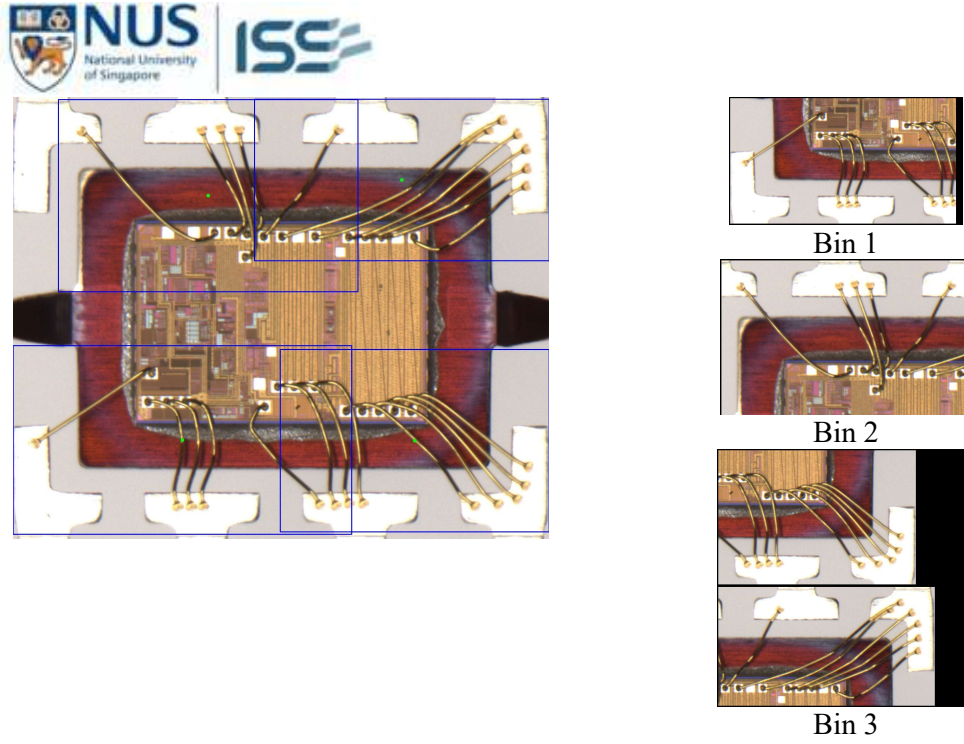
Dissimilarity function between two boxes:

$$D(a, b) = \begin{cases} wh & \text{where } w < 1008 \text{ and } h < 1008 \\ REALMAX & \text{otherwise} \end{cases}$$

Linkage function between two cluster:

$$L(A, B) = \begin{cases} \min \{D(a, b), a \text{ in } A, b \text{ in } B \text{ if } W < 1008 \text{ or } H < 1008\} \\ REALMAX & \text{otherwise} \end{cases}$$

Since image patches often measure less than 1008x1008, they are consolidated into standardized 1008x1008 bin images using a bin packing algorithm. Because the algorithm's efficiency is highly sensitive to the input sequence, a genetic algorithm (GA) is implemented to optimize the packing order. Each chromosome consists of genes representing an image patch's sequence ID and a 90-degree rotation flag. The fitness function $\sum_i u_i^2 / N^2$ is designed to maximize bin utilization u while minimizing the total number of bins N required for the dataset.



(a) Hierarchical Clustering Result (b) GA based bin packing results
Figure 9. Partition and Merge Result

The SAM 3 binary instance masks are converted into polygonal approximations using the OpenCV library. To restore the spatial integrity of the data, the local coordinates of these polygons are remapped to the original image coordinate system by accounting for their respective patch offsets and 90-degree rotation flags.

3.3.2 Exemplar Recommender

Exemplar selection is critical to the predictive performance of SAM 3. To facilitate this, we constructed a recommendation database containing image data and exemplar Regions of Interest (ROIs). Because SAM 3 accuracy is positively correlated with the visual similarity between input and training exemplars, we quantify this relationship using cosine similarity between their respective feature embeddings. These embeddings are generated by the SAM 3 image encoder, which produces $72 \times 72 \times 256$ visual features. Finally, global average pooling is applied to compress these features into a robust $1 \times 1 \times 256$ vector for efficient similarity matching.

The recommendation database is built from a curated dataset, where exemplars are derived directly from the images designated for validation. To establish the initial exemplar set, wires exhibiting the minimal spatial overlap with neighboring structures are selected (See Fig. 11). These candidates are subsequently verified and refined through a manual audit of the SAM 3 prediction results to ensure high-fidelity ground truth.

In its original configuration, SAM 3 requires exemplars to be selected from the same image being processed for instance segmentation. As illustrated in the SAM 3 detector architecture (Fig. 10), the exemplar encoder transforms reference samples into feature embeddings for both

the prompt and the prompt mask. We have modified the detector to decouple this encoder from the primary architecture. This modularity allows the exemplar to be pre-encoded and its embeddings cached; these stored values are then fed directly into the multimodal decoder to perform instance segmentation across different images.

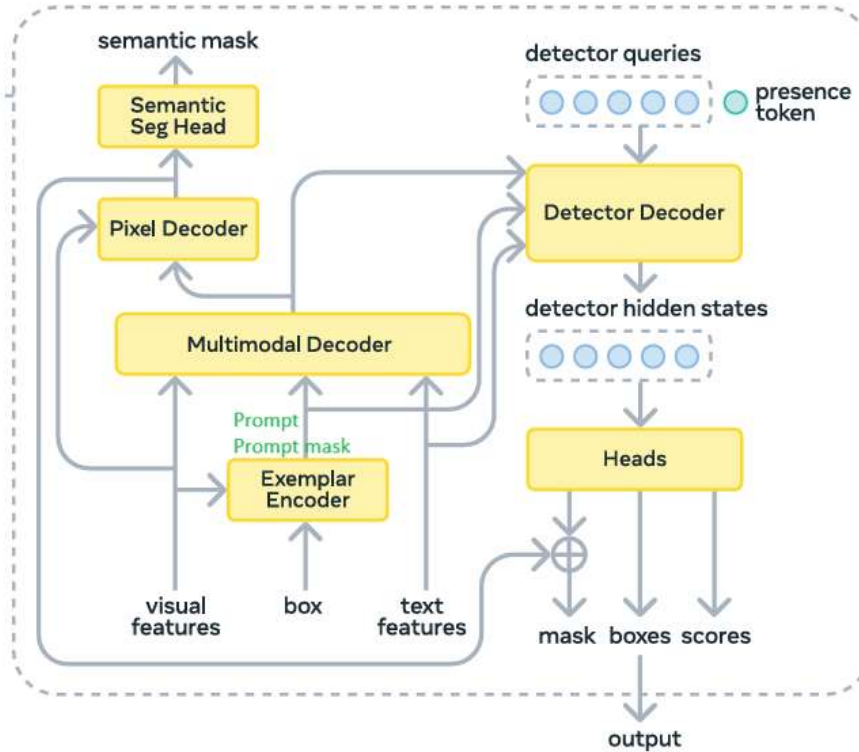


Figure 10. SAM3 Detector Architecture

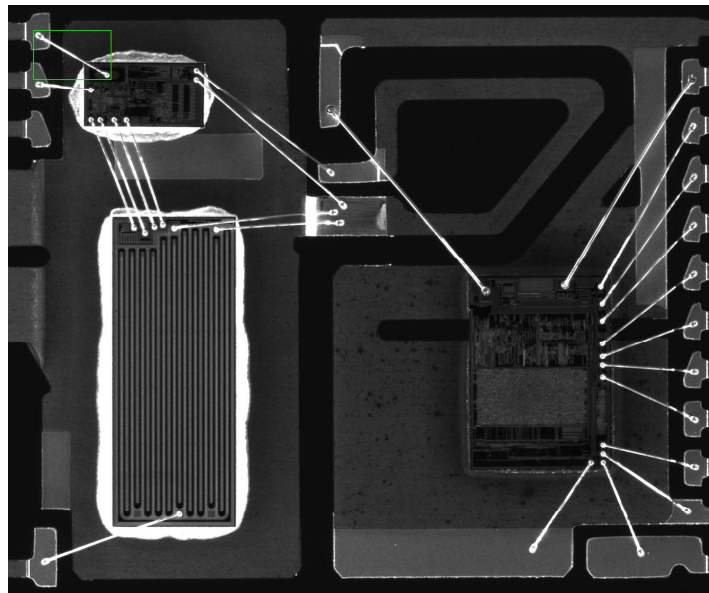


Figure 11. Exemplar image and ROI

3.3.3 SAM3 Finetune

Fine-tuning of the SAM 3 model is conducted using the official Meta GitHub repository as the framework. To accommodate the specific requirements of the wire bonding dataset, the data loader (See Fig. 12) was customized: a dedicated data reader converts raw images and annotations into image tensors and wire instance masks. We implemented an anchor-based sampling strategy that randomly selects a target wire to serve as the center for a 1008x1008 crop to ensure training efficiency. These cropped image-mask pairs then undergo a suite of transformations via a data augmenter before being fed into the model for training.

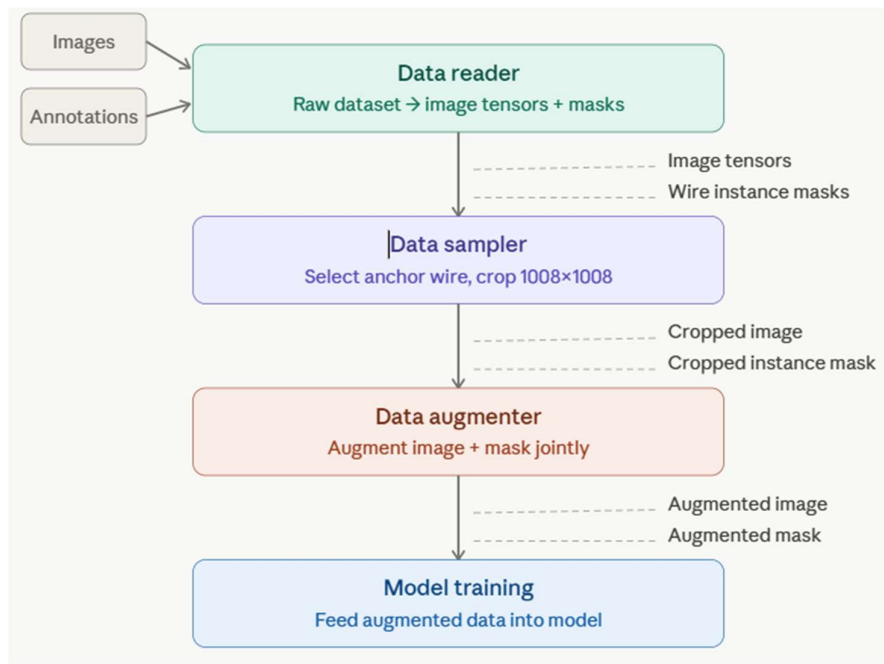


Figure 12. Model finetune Data Loader

3.3.4 Label Error Identification

To identify label errors, we utilize Intersection over Union (IoU) and Hausdorff distance as primary evaluation metrics. Using the Hungarian matching algorithm, each wire annotation is paired with a corresponding prediction based on their spatial overlap. A correspondence is rejected if the IoU is below 0.5. For successfully matched pairs, the annotation is flagged as a label error if the IoU falls below a secondary threshold or if the Hausdorff distance exceeds a predefined limit. Additionally, any unmatched annotations or predictions are automatically categorized as label errors.

3.4 Result Merging

Redundancies and conflicts exist among the label errors identified by the three methods. To resolve this, label error prediction is governed by the following equation, utilizing a logical OR approach: a label is categorized as an error if it is flagged by at least one of the three methods. This equation assumes that all three methods minimize false positives to the greatest extent possible.

$$\bar{p} = \sum_{i=1}^3 p_i > 0$$

4. System Implementation

4.1 Data Preparation

The raw wire bonding dataset are collected from RoboFlow Universe[10]. The dataset annotation is in MSCOCO segmentation format [11].

Database Name	Image Number	Image Number after filtering
detr-large-scale-dbwde	139	90
dos-semicon-e4lob	41	41
fariq-generalize-dos-r02td	39	35
unisem-zn1461-avrrj	50	40

Table 2. Database Summary

Because the MSCOCO format stores all annotations in one file, performing image-by-image quality reviews is difficult. We addressed this by converting the dataset into a folder-based structure where every image is paired with its own JSON file. During this process, we filtered out images containing highly dense wires, as they proved too complex for effective instance segmentation.

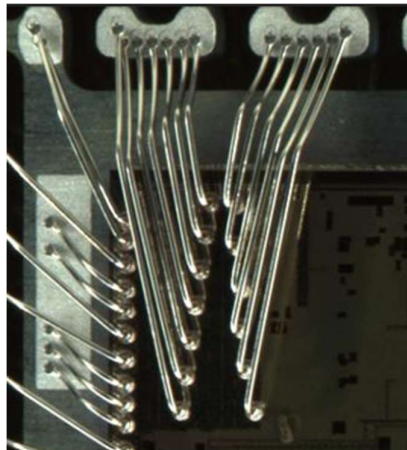


Fig. 13 Highly dense wire image

Observation revealed that the default wire annotations lack the two essential bond points (see Fig. 15(a)). To rectify this, the complete dataset was curated through an active learning framework, as illustrated in Fig. 15. In each iteration, the top 5% of samples with the lowest prediction scores were selected for refinement. As shown in Fig. 16, the bond points are included in wire mask after fully data curation.

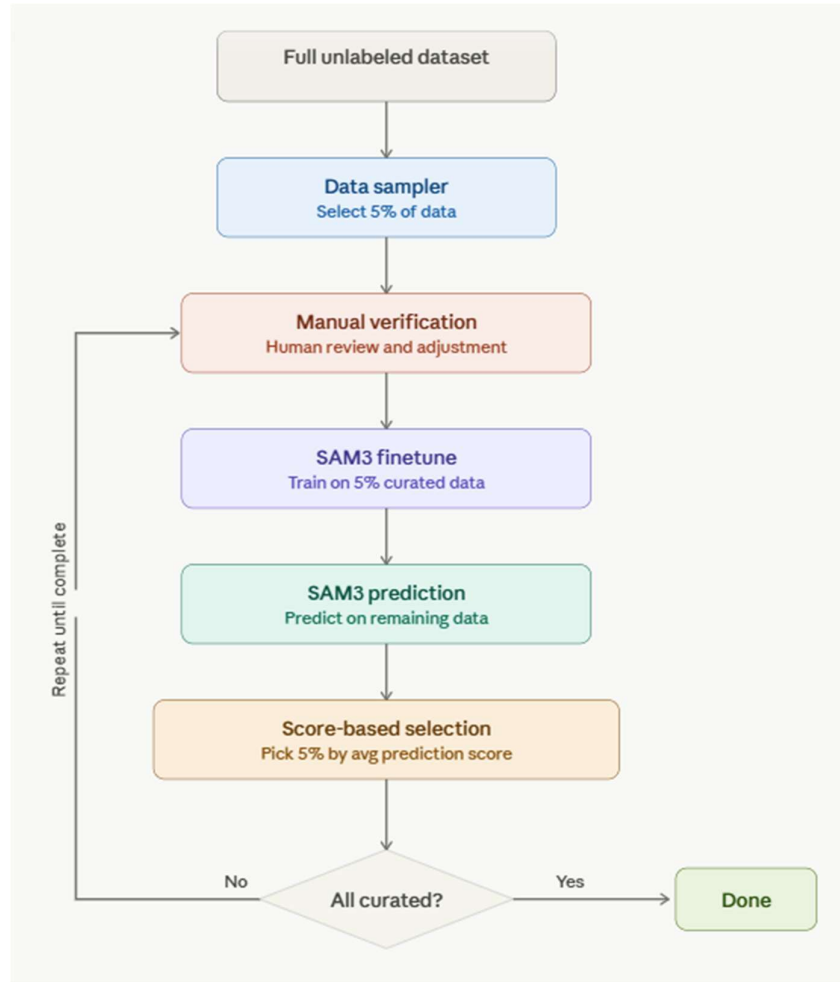


Fig. 15 Active learning based data curation

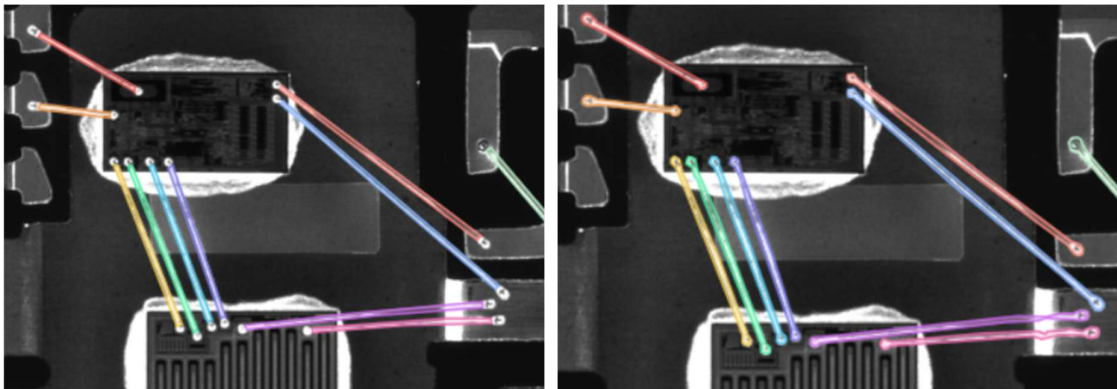


Fig. 16 Before and after data curation

4.1.1 Synthetic Data Generation

There is no ground truth of label errors in database. It is impossible to verify the whole system performance. The dataset with synthetic label errors are generated as shown in Table 3.

Error	Description	Error Number
Undefined label name		114
Undefined shape type		357
Image out of focus	Apply Gaussian Smoothing	90
Low contrast Image	$I = (I - \text{mean}) * \text{random}(0.1, 0.3) + \text{mean}$	132
Shape points out of image range		362
Inaccurate boundary error	Apply small rotation, scaling to wire masks	739

Table 3. Summary of synthetic errors

4.2 JSON Validator

To identify JSON-related errors, we defined both a label schema (see Fig. 17) and a shape schema (see Fig. 7). While the jsonschema Python library is used for validation, applying both schemas simultaneously to the entire file results in overly verbose error messages that obscure the specific source of the failure. To resolve this, we validate each label individually against both schemas. If an exception is raised during a specific iteration, that label is immediately flagged as an error.

```

LABEL_SCHEMA = {
    "type": "object",
    "required": [
        "Label",
        "Shapes"
    ],
    "properties": {
        "Label": {
            "type": "string",
            "enum": [
                "wire"
            ]
        },
        "Shapes": {
            "type": "array",
            "minItems": 1
        },
        "additionalProperties": True
    }
}

```

Fig. 17 Label Schema

4.3 Feature based classifier Validator

We conducted histogram analysis on all features (see Fig. 18) using red for error and blue for normal labels. Because hue and saturation showed zero differentiation power, they were removed from the feature set. Since no individual feature could perfectly separate the classes, we trained a decision tree model from scikit-image library[12] using a 1:100 class weight ratio to avoid false alarm. This configuration resulted in high precision but lower recall. Such result also simplifies the merging process. The synthetic dataset is randomly split into training and testing at ratio of 50:50.

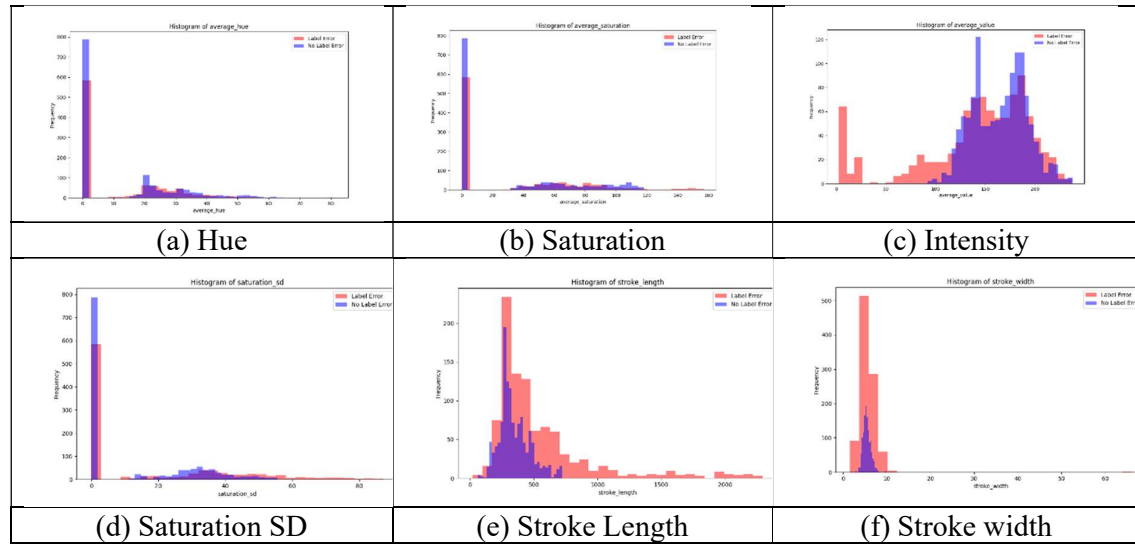


Fig. 18 Histogram Analysis of features

4.4 SAM3 based Validator

The SAM3 is finetuned using below configuration with customized data loader.

Echo	300
LoRA	r: 32 alpha: 32 drop out: 0.1
Optimizer	AdamW
Learning rate	1e-5
Data augmentation	Rotation: [-45,45] Shear: [-10,10,-10,10] Translation: [0.3, 0.3] Scale: [0.8,1.2]

Table 4. SAM3 training configuration

The SAM3 detector outputs instance bounding box, instance classification, instance masks and semantic masks. In original settings the GIOU and L1 loss are used for box, the focal loss is used for instance classification, dice and focal loss are used for instance mask and semantic mask. However, the weight of each loss is adjusted to achieve better performance.

Loss	Original Weight	Adjusted Weight
Box GIOU	2.0	4.0
Box L1	5.0	50.0
Classification focal	20.0	2.0
Instance Segmentation focal	10.0	5.0
Instance Segmentation Dice	200.0	600.0
Semantic Segmentation focal	20.0	20.0
Semantic Segmentation Dice	30.0	30.0

Table 5. SAM3 training loss weight

The wire IOU on test dataset is improved from 0.7 to 0.89.

4.5 System Evaluation

The data validation is summarized into a JSON result file for later display in GUI.

```
{
  "Labels with Issue": [
    {
      "id": 0,
      "data failure reason": "SAM3 Va",
      "issue_image": "test_cases/Robo1
      ...
    },
    ...
  ],
  "inspection_path": "test_cases/RoboFlow",
  "False Positives": [],
  "False Negatives": [],
  "True Positives Num": 11,
  "True Negatives Num": 13,
  "False Positives Num": 0,
  "False Negatives Num": 0,
  "Precision": 1.0,
  "Recall": 1.0
}
```

Fig. 19 Result Summary JSON File

For each label with issue, false positives, and false negatives, the resulting JSON includes the category_id, data_fail_reason, and issue_image. Validation errors are populated based on the detecting source: the JSON schema validator, the feature-based classifier, or the SAM3-based validator. If multiple methods flag a single label, their respective error messages are concatenated. Furthermore, an issue_image is generated by overlaying the wire boundary with the SAM3 prediction (where available) and cropping the result to the wire boundary's bounding box.

The system performance is evaluated by using resulting merging equations. The precision and recall are reported for every method and combination of all of them in Table 6. It can be observed that combination of all methods delivers the best performance.

Method	Train Precision and Recall	Test Precision and Recall
JSON Validator	1.0 and 0.52	1.0 and 0.56

Feature-based Classifier Validator	1.0 and 0.36	1.0 and 0.31
SAM3 based Validator	0.95 and 0.95	0.94 and 0.93
All	0.97 and 0.97	0.97 and 0.98

Table 6. System Performance Summary

The partition and merging efficiency are evaluated by number of bins divided by number of bins generated by even partition with 10% of overlapping. The average ratio is roughly 2.1 which means half of SAM3 inference time are saved.

Despite the high overall performance, the system still encounters a significant volume of false positives and false negatives. These discrepancies primarily stem from SAM3's sensitivity, where marginal differences between the ground-truth annotations and SAM3 predictions trigger error flags. Given that the classification of errors relies on fixed IoU and Hausdorff distance thresholds, eliminating these borderline cases remains a significant technical challenge.

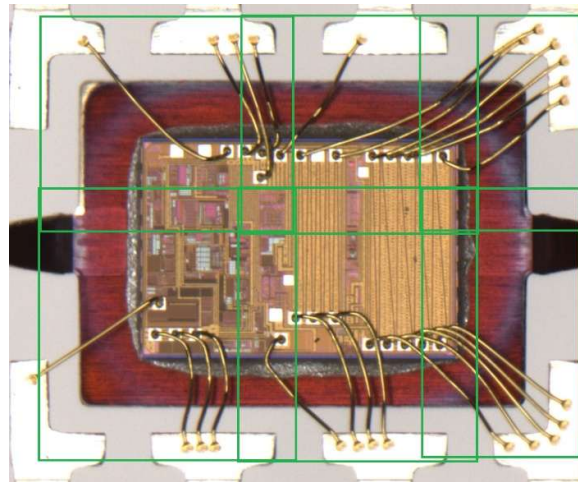


Figure 20. Even partition

4.6 System GUI

The system GUI is developed using Gradio framework. User could selected data points and methods for validation from the drop down list. After selection user could click on start validation and wait for validation summary. The validation summary consists of label with issues. Every label with issues comes with image and data failure reason. The false positives and false negatives are also listed below in case the ground truth is known.

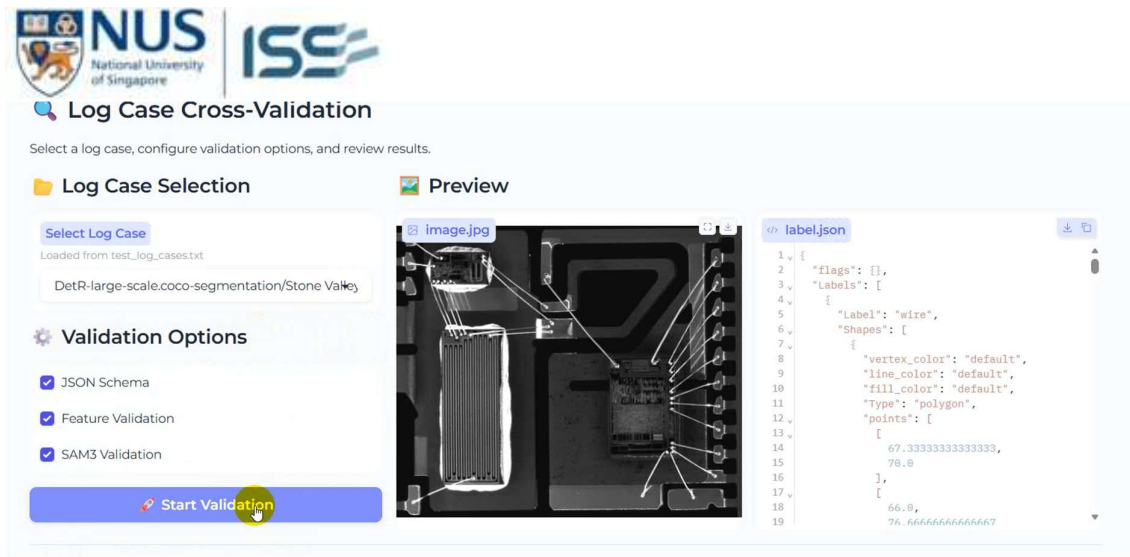


Figure 21. GUI

5. Conclusion

5.1 Summary

In conclusion, we have developed an integrated validation system designed to identify and rectify errors in instance segmentation labels. The framework leverages schema-based verification, feature analysis, and SAM3-based geometric validation to achieve high-precision and recall results. This multi-layered approach overcomes the limitations of monolithic annotation formats, providing a scalable solution for maintaining data integrity in complex visual inspection tasks.

5.1 Limitations

Currently, the system supports only a single exemplar. However, as noted in the SAM3 literature, providing additional exemplars generally yields superior performance. While the current single-exemplar configuration produces acceptable results for datasets sourced from Roboflow Universe, we anticipate that incorporating multiple exemplars will be necessary to maintain high performance when encountering more complex wire bonding packages or new data distributions.

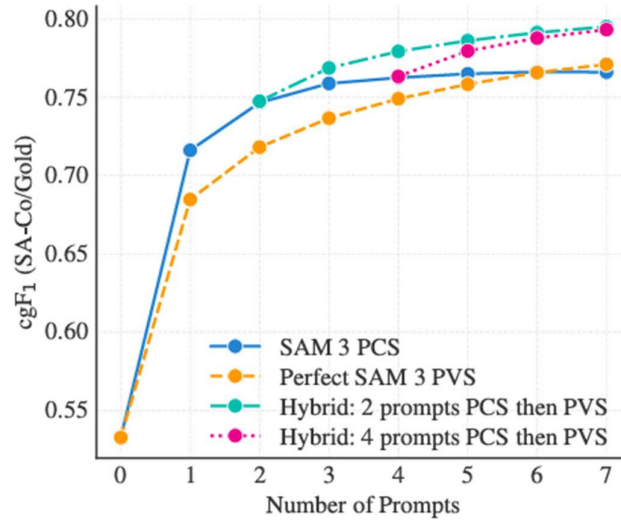


Figure 21. Number of Prompts vs. F1 (Quoted from SAM3)

In addition to the errors identified in Table 1, future iterations should include labels for wire defects like broken wires and lifted bonding. These labels are critical for training the defect classification models used in AOI outputs. It should be noted that our current three-method validation framework lacks the capacity to fully detect and verify errors for these complex defect types.

5.3 Future work

Future developments will focus on enhancing the diagnostic capabilities of the system by integrating multimodal large language models (MLLMs), such as Gemma. By leveraging Gemma's ability to process both visual and textual data, the system could provide nuanced, natural-language explanations for detected errors, moving beyond simple failure flags to actionable insights for human reviewers.

Furthermore, expanding the system to encompass functional wire defects—such as broken or lifted bonds—is a priority. To address the inherent lack of labeled defect data, we propose implementing unsupervised anomaly detection frameworks like PatchCore[14] to identify structural deviations. Alternatively, an approach utilizing high-dimensional feature embeddings followed by a specialized classifier could be employed to categorize specific defect types.

The SAM3 model is currently fine-tuned exclusively on a 'normal' dataset, characterized by the absence of structural defects. Consequently, its performance may be suboptimal when processing out-of-distribution data containing anomalies. Given the inherent scarcity and difficulty of collecting real-world defective samples, generative AI presents a viable solution. Specifically, state-of-the-art image generation models like Nano Banana Pro[15] can be leveraged to synthesize realistic defect simulations. Integrating these synthetic anomalies into the training pipeline would allow the model to learn a more robust decision boundary, ultimately enhancing its predictive accuracy on complex, real-world datasets.

- [1] Cleanlab Inc. (2026). *Cleanlab: The standard data-centric AI package for data quality and machine learning with messy, real-world data and labels* (Version 2.6) [Computer software]. GitHub. <https://github.com/cleanlab/cleanlab>
- [2] Moore, B. E., & Corso, J. J. (2025). *Voxel51's FiftyOne* (Version 1.7.0) [Software]. <https://voxel51.com/>
- [3] CVAT.ai Corporation. (2026). *Computer Vision Annotation Tool (CVAT)* (Version 2.64.0) [Computer software]. GitHub. <https://doi.org/10.5281/zenodo.18925818>
- [4] V7 Labs. (2026). *V7 Darwin: AI Data Labeling & ML Training Data Platform* (Version 3.5) [Software]. <https://www.v7labs.com/darwin>
- [5] SuperAnnotate. (2026). *SuperAnnotate: Bringing Human Intelligence to AI* (Version 4.10) [Software]. <https://www.superannotate.com/>
- [6] Sapkota, R., Cheppally, R. H., Sharda, A., & Karkee, M. (2025). YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection. *arXiv preprint arXiv:2509.25164*. <https://doi.org/10.48550/arXiv.2509.25164>
- [7] Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., Assran, M., Ballas, N., Galuba, W., Howes, R., Huang, P.-Y., Li, S.-W., Misra, I., Rabbat, M. G., Sharma, V., ... Bojanowski, P. (2023). DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*. <https://arxiv.org/abs/2304.07193>
- [8] Mazzini, D., Ravi, N., Reizenstein, J., Girdhar, R., Joulin, A., & Carion, N. (2025). *SAM 3: Segment Anything with Concepts*.
- [9] van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & the scikit-image contributors. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>
- [10] Dwyer, B., Nelson, J., & Solawetz, J. (2024). *Roboflow (Version 1.0)* [Software]. Available from <https://roboflow.com>.
- [11] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Computer Vision – ECCV 2014* (pp. 740–755). Springer International Publishing. https://doi.org/10.1007/978-3-319-10602-1_48
- [12] van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., Yu, T., & scikit-image contributors. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>
- [13] Gemma Team. (2024). *Gemma: Open models based on gemini research and technology*. arXiv preprint arXiv:2403.08295. <https://doi.org/10.48550/arXiv.2403.08295>

[14] Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 14318–14328).

[15] Google. (2026). *Nano Banana 2 (Gemini 3 Flash Image)* [Large-scale generative image model]. Google AI. <https://gemini.google.com>

AI Tool Use Declaration

While the overarching system architecture and core conceptual logic were developed entirely by the author, several AI tools were utilized to streamline the implementation phase. **GitHub Copilot** provided real-time assistance during the coding process, and **Claude Code** was employed to generate the initial frameworks for the annotation viewer and Gradio UI, which were subsequently refined through manual modifications. Finally, **Gemini** was leveraged for syntax verification.

Appendix A Project Proposal

GRADUATE CERTIFICATE: Intelligent Reasoning Systems (IRS)

Date of proposal: 9 April 2026
Project Title: Data Quality Assurance System for Wire Bonding Inspection
Sponsor/Client: <i>(Name, Address, Telephone No. and Contact Name)</i> Institute of Systems Science (ISS) at 25 Heng Mui Keng Terrace, Singapore NATIONAL UNIVERSITY OF SINGAPORE (NUS) Contact: Dr. Gary Leung Email: cc.leung@nus.edu.sg
Background/Aims/Objectives: To support safety-critical wire-bonded IC inspection, this project develops a data quality assurance system to detect and reduce image/annotation errors in AOI datasets, improving software validation reliability and deep-learning model performance.
Requirements Overview: • Research ability • Programming ability • System integration ability
Resource Requirements (please list Hardware, Software and any other resources) Hardware proposed for consideration: • GPU, etc. Software proposed for consideration: • Machine Learning Models: SAM 3 / Segment Anything Model 3 (SAM3) • Training platform: Pytorch

- Inference platform: Triton
- Image processing toolbox: OpenCV, Scikit-image
- JSON schema validation: jsonschema
- Annotation reader and writer: JSON
- GUI: Gradio

Number of Learner Interns required: (Please specify their tasks if possible)

A team of two project members

Methods and Standards:		
Procedures	Objective	Key Activities
Requirement Gathering and Analysis	The team should meet with ISS to scope the details of project and ensure the achievement of business objectives.	1. Gather & Analyze Requirements 2. Define internal and External Design 3. Prioritize& Consolidate Requirements 4. Establish Functional Baseline
Technical Construction	● To develop the source code in accordance to the design. ● To perform unit testing to ensure the quality before the components are integrated as a whole project	1. Setup Development Environment 2. Understand the System Context, Design 3. Perform Coding 4. Conduct Unit Testing
Integration Testing and acceptance testing	To ensure interface compatibility and confirm that the integrated system hardware and system software meets requirements	1. Prepare System Test Specifications 2. Prepare for Test Execution 3. Conduct System Integration Testing 4. Evaluate Testing

	and is ready for acceptance testing.	5. Establish Product Baseline
Acceptance Testing	To obtain ISS user acceptance that the system meets the requirements.	1. Plan for Acceptance Testing 2. Conduct Training for Acceptance Testing 3. Prepare for Acceptance Test Execution 4. ISS Evaluate Testing 5. Obtain Customer Acceptance Sign-off
Delivery	To deploy the system into production (ISS standalone server) environment.	1. Software must be packed by following ISS's standard 2. Deployment guideline must be provided in ISS production (ISS standalone server) format 3. Production (ISS standalone server) support and troubleshooting process must be defined.

Team Formation & Registration

Team Name:

Vision Guard

Project Title(repeated):

Data Quality Assurance System for Wire Bonding Inspection

System Name (if decided):

Team Member 1 Name: **Zheng Bo**

Team Member 1 Matriculation Number: **A0339761U**

Team Member 1 Contact (Mobile/Email): **zhengbo@u.nus.edu**

Team Member 2 Name: Sun Yan
Team Member 2 Matriculation Number: A0340238L
Team Member 2 Contact (Mobile/Email): Sun.Yan@u.nus.edu

Appendix B: Mapped System Functionalities against knowledge, techniques, and skills of modular courses: MR, RS, CGS

In the proposed project, we have integrated and demonstrated multiple aspects from the following four technique groups:

1. Decision Automation

The validation pipeline utilizes schema enforcement via jsonschema to ensure structural integrity. The decision tree classifier serves as a reasoning engine, utilizing a series of "If-Then" conditions to automate error labelling. The fine-tuned SAM3 model contributes by automating complex knowledge-based judgments of wire boundaries accuracy, effectively functioning as a rule-based component within whole pipeline. This automation is further enhanced by threshold-based reasoning, where specific IoU and Hausdorff distance metrics programmatically verify SAM3 predictions against existing annotations.

2. Business Resource Optimization

The active learning curates the "full data" by isolating the top 5% of samples with the lowest prediction scores, thereby optimizing the limited resource of human review time. Human annotator only needs to pay more attention to the most uncertain or high-value data points. Furthermore, the implementation of feature selection via histogram analysis identifies and removes variables with "zero differentiation power," such as hue and saturation, which significantly optimizes the computational resources required for the decision tree classifier to maintain high-speed performance.

The partition and merge minimizes the number of input images for SAM3 inference. Hence, the GPU resource consumption is optimized.

3. Knowledge Discovery & (Big) Data Mining

Through SAM3 fine-tuning, the model undergoes domain adaptation to "discover" the specific geometric knowledge of the wire bonding. This fine-tuned model then performs wire segmentation to identify discrepancies between human-generated labels and model predictions. Simultaneously, the exemplar recommender utilizes collaborative filtering to mine historical data and "discover" similarities between current packages and successful past cases. This predictive capability allows the system to recommend high-quality exemplars.

4. Cognitive Techniques or Tools

The Gradio UI serves as data validation in an intuitive format that allows human operators to interpret results and act upon them effectively. This is further enhanced through visual communication via the automated generation of "issue images," which overlay SAM3 predictions directly onto wire images to provide a natural, human-centric diagnostic rather than a standard numerical error code.

Appendix C: Installation and User Guide

1. Prerequisite

- Python 3.12 or higher
- PyTorch 2.7 or higher
- GPU driver ≥ 580
- CUDA-compatible GPU with CUDA 12.6 or higher with at least 11GB memory
- OS: Ubuntu 24.04 (Windows not supported)

2. Installation

1. Create new conda environment

```
conda create -n data_validation python=3.12
```

```
conda deactivate
```

```
conda activate data_validation
```

2. Install pytorch with CUDA support

```
pip install torch==2.10.0 torchvision --index-url https://download.pytorch.org/whl/cu128
```

3. Clone or download the code repo

```
cd SystemCode
```

4. Download **sam_ft.pt** from <https://drive.google.com/file/d/10axKumAmnKISg-bz39DhfWZDdz19j4su/view?usp=sharing> to folder **checkpoints\finetuned** (about 3.2GB)

5. Download dataset from

https://drive.google.com/file/d/1W8ETU_Zt2UdVPyX9PVMRIYjgOg9Yrb8/view?usp=sharing and unzip two folders RoboFlowConverted and RoboFlowSyntheticData to folder **test_cases**

6. Install other dependencies

```
pip install requirements.txt
```

7. Install gradio

```
Pip install gradio
```

3. User guide

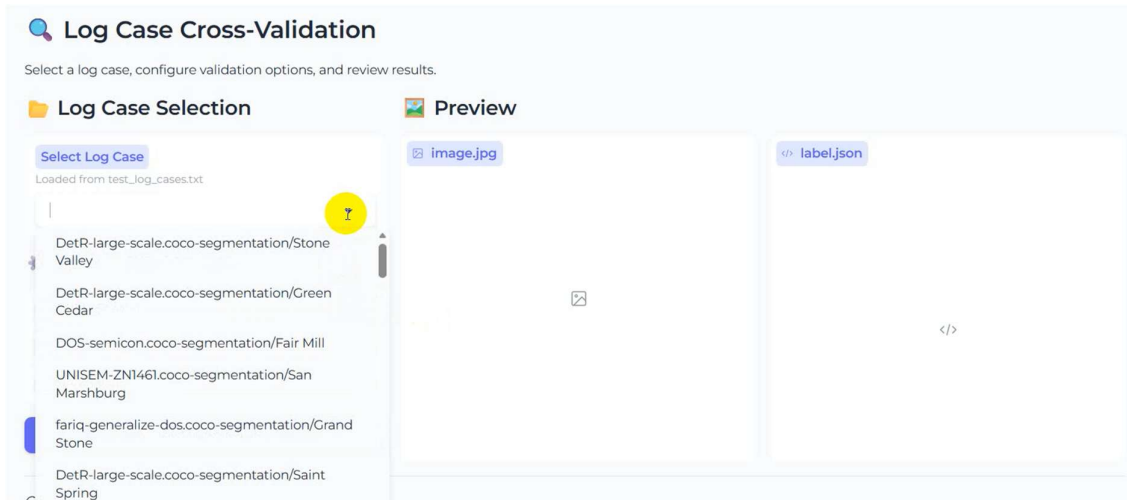
1. Start the GUI

```
python run_data_validation_demo.py
```

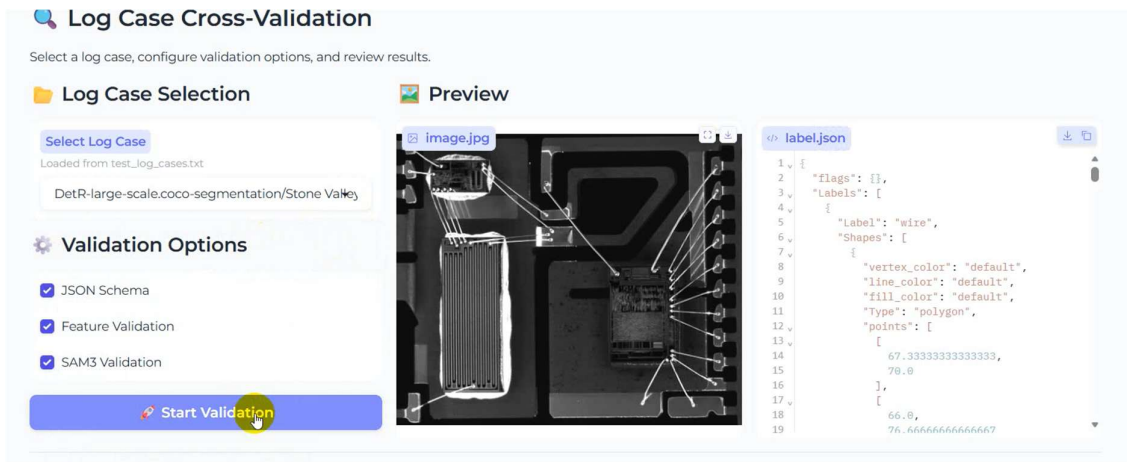
2. Open the link when below message is observed

```
ision/zhengbo/DataCleanEngine/run_data_validation_demo.py
Foundation model loaded
Knowledge graph built s Follow link \(ctrl + click\)
Running on local URL: http://0.0.0.0:7850
To create a public link, set `share=True` in `launch()`.
```

3. Select the data for validation from the drop down list



4. Click on “Start Validation” button



5. Review the validation summary

Validation Summary

9	13	1	0	90.0%	100.0%
True Positives	True Negatives	False Positives	False Negatives	Precision	Recall

11	1	0
Labels with Issue	False Positives	False Negatives

Inspection Path test_cases/RoboFlowSyntheticData/DefR-large-scale.coco-segmentation/Stone Valley

Labels with Issue

ID	Issue Image	Data Failure Reason
1		SAM3 Validator: Unmatched annotation, no corresponding prediction found.